

Abstract Factory Pattern

Justification

The design pattern chosen for the catalog search functionality is the Abstract Factory.

The Abstract Factory Pattern fits the Library Management System because the search feature is not a single operation — it is a family of related behaviors that vary by search type. When a user searches by title, the system needs a query builder that targets the title field and a formatter that presents results grouped by title. When searching by author, the query targets a different field and the output may be ordered alphabetically by last name. When searching by ISBN the query is an exact match rather than a keyword scan. Each search type forms a coherent family of two related objects: a `SearchQuery` and a `ResultFormatter`. The Abstract Factory separates the creation of these families from the code that uses them.

The `SearchContext` class holds a reference to a `SearchFactory` interface and calls `createSearchQuery()` and `createResultFormatter()` at runtime. Because `SearchContext` depends on the abstraction rather than any concrete class, swapping the entire search family requires a single call to `setFactory()`. No conditional logic is scattered across the codebase and no existing class needs to be modified when a new search type, such as a genre-based search or a keyword search, is introduced. A new factory class implementing `SearchFactory` is added and the rest of the system is untouched.

This design satisfies the SOLID principles that the system must follow. The Open/Closed Principle is respected because adding `GenreSearchFactory` requires no changes to `SearchContext` or any existing factory. The Single Responsibility Principle is enforced because each factory is responsible only for producing one consistent search family. The Dependency Inversion Principle is satisfied because high-level logic depends on the `SearchFactory` interface, not on `TitleSearchFactory` or any other concrete class. The Liskov Substitution Principle holds because any concrete factory can be placed in the context and the system behaves correctly.

Compared to the Strategy pattern, which the team originally considered, Abstract Factory is the better fit here. Strategy manages a single interchangeable algorithm. Abstract Factory manages a coordinated family of objects that must match each other. A title query built by `TitleSearchFactory` must be paired with a title-aware formatter, not with the formatter produced by `ISBNSearchFactory`. The factory enforces that pairing by construction. Strategy provides no such guarantee.

This pattern is also directly traceable to the functional requirement BR_LMS_07, which states that the system shall allow users to search for books using keywords, and BR_LMS_33, which requires advanced filters for faculty. The factory structure makes it straightforward to introduce a FacultySearchFactory that applies academic filters without touching the student-facing search logic.

UML Class Diagram

